CS 5402: Introduction to Data Mining

SP2026

(Lecture - 14)

Dimensionality Reduction & t-SNE

Dr. Huiyuan Yang

Department of Computer Science
Missouri S&T

https://hyang428.github.io

MISSOURI S&T

Motivation

Why Dimensionality Reduction?

Modern datasets often have:

- Hundreds or thousands of features
- High storage cost
- Slow computation
- Visualization difficulty
- Curse of dimensionality

Goal:

- Represent high-dimensional data in fewer dimensions while preserving structure.

The Curse of Dimensionality

- Real-world data often exists in hundreds or thousands of dimensions (e.g., pixels in an image, word counts in a document).
- The *curse of dimensionality* refers to various phenomena that arise when analyzing and organizing data in high-dimensional spaces that do not occur in low-dimensional settings:
  - *The volume of the space increases so fast that the available data become sparse*. In order to obtain a reliable result, the amount of data needed often grows exponentially with the dimensionality.
  - Organizing and searching data often relies on detecting areas where objects form groups with similar properties; *in high dimensional data, however, all objects appear to be sparse and dissimilar in many ways*, which makes common data organization strategies less efficient.

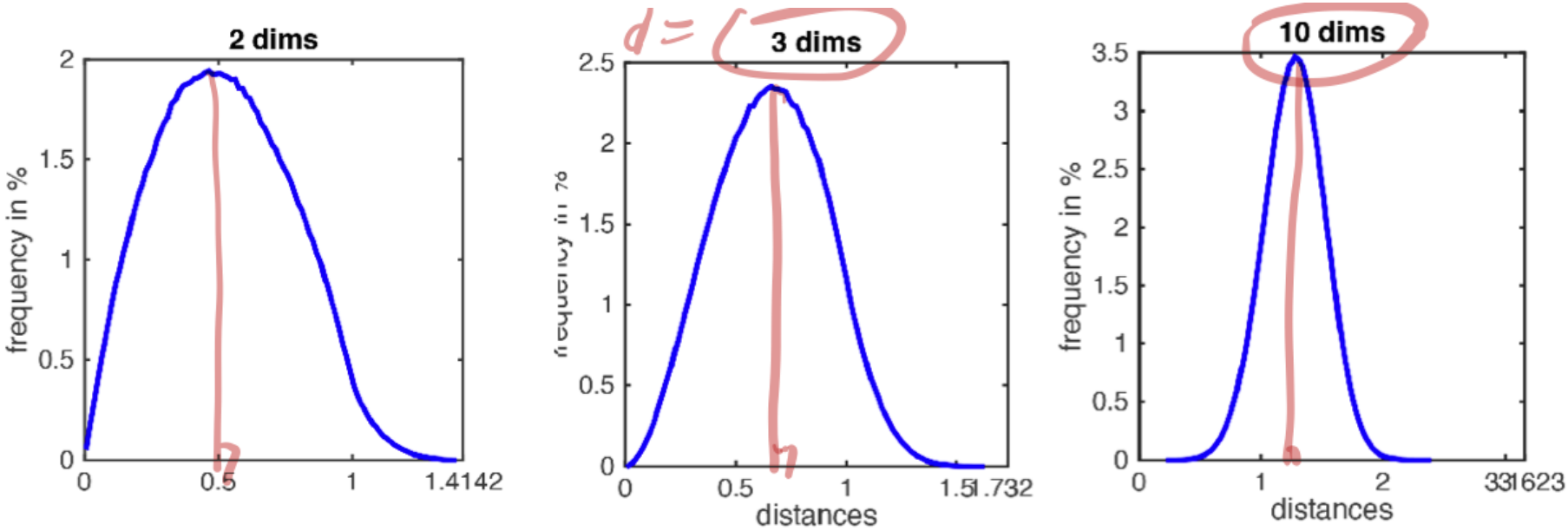
Curse of Dimensionality Intuitive Explanation (1)

- Claim: The distance between two sampled points increases as d grows.

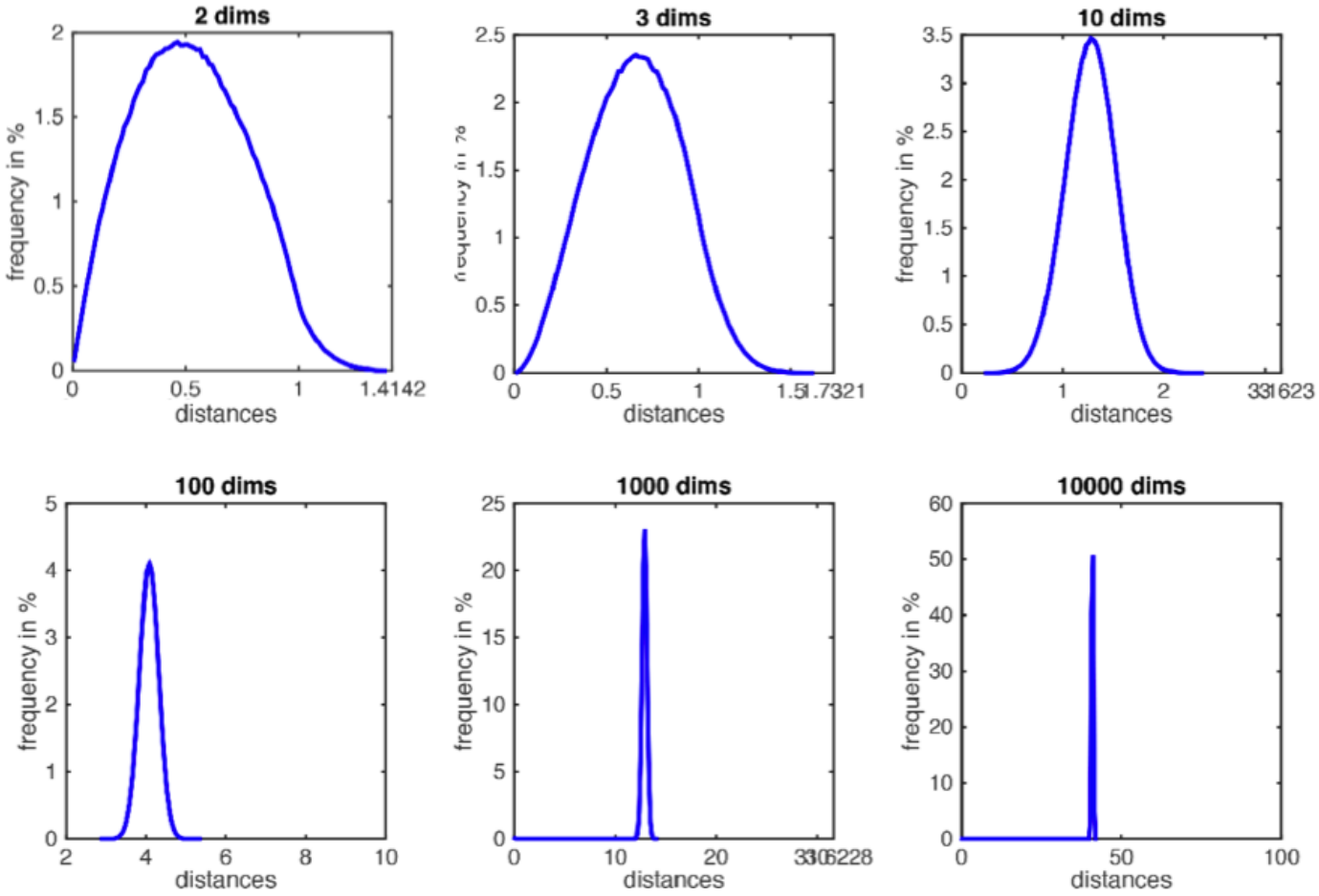
Example: let us consider uniform distribution over a cube $[0, 1]^d$, we uniformly sample two points x_1, x_2, and calculate the distance:

$d(x_1, x_2) = ||x_1 - x_2||_2$

Let's plot the distribution of such distance:



Curse of Dimensionality Intuitive Explanation (2)

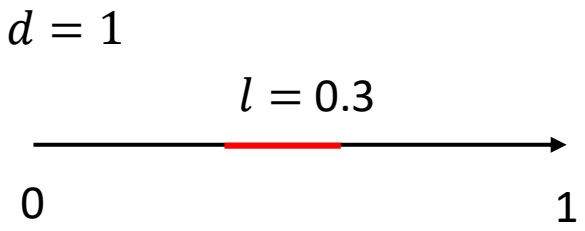


Distance increases as $d \rightarrow \infty$

Curse of Dimensionality Intuitive Explanation (3)

- Key problem: in high dimensional space, points that are draw from a distribution tends to be far away from each other!

Example: let us consider uniform distribution over a cube $[0, 1]^d$



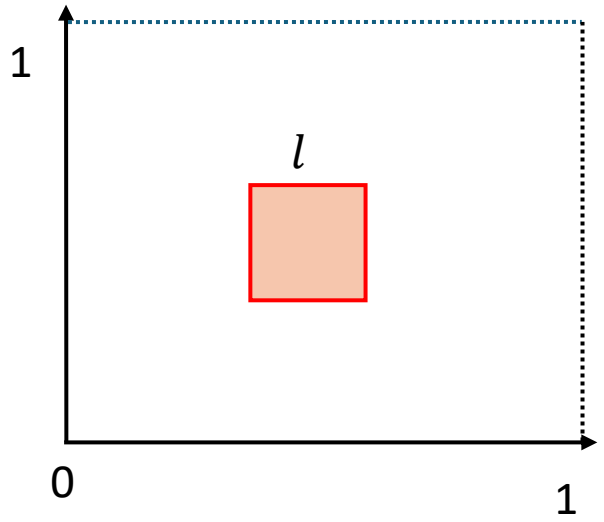
Q: sample *x* uniformly, what is the probability that *x* is inside the small cube?

$$\frac{l}{1} = l$$

Curse of Dimensionality Intuitive Explanation (4)

- Key problem: in high dimensional space, points that are draw from a distribution tends to be far away from each other!

Example: let us consider uniform distribution over a cube $[0, 1]^d$



$d = 2$

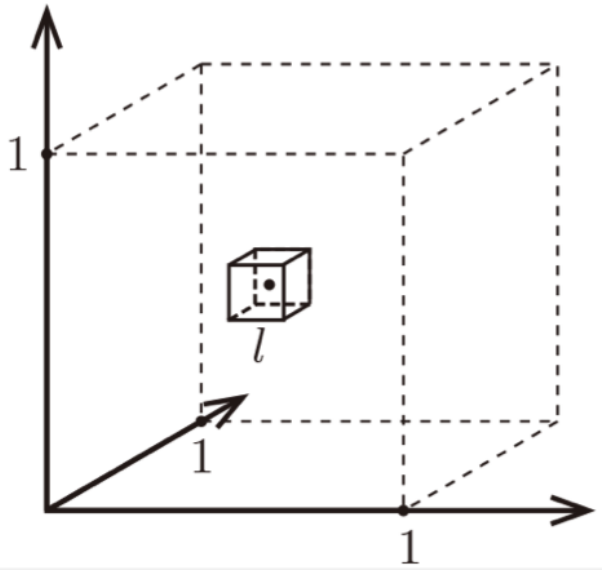
Q: sample x uniformly, what is the probability that x is inside the small cube?

$$\frac{l^2}{1} = l^2$$

Curse of Dimensionality Intuitive Explanation (5)

- Key problem: in high dimensional space, points that are draw from a distribution tends to be far away from each other!

Example: let us consider uniform distribution over a cube $[0, 1]^d$



$d = 3$

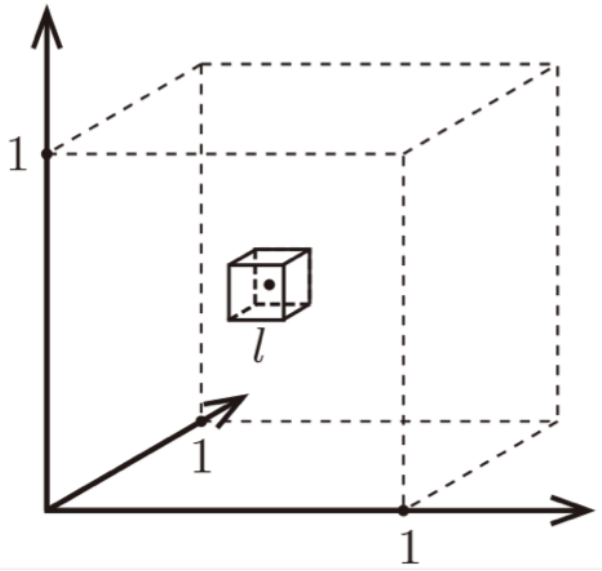
Q: sample *x* uniformly, what is the probability that *x* is inside the small cube?

$$\frac{l^3}{1} = l^3 \quad \rightarrow \quad l^d$$

Curse of Dimensionality Intuitive Explanation (6)

- Key problem: in high dimensional space, points that are draw from a distribution tends to be far away from each other!

Example: let us consider uniform distribution over a cube $[0, 1]^d$



$d = 3$

Now assume we sample *n* points uniform randomly in the big cube, and we observe *K* points fall inside the small cube

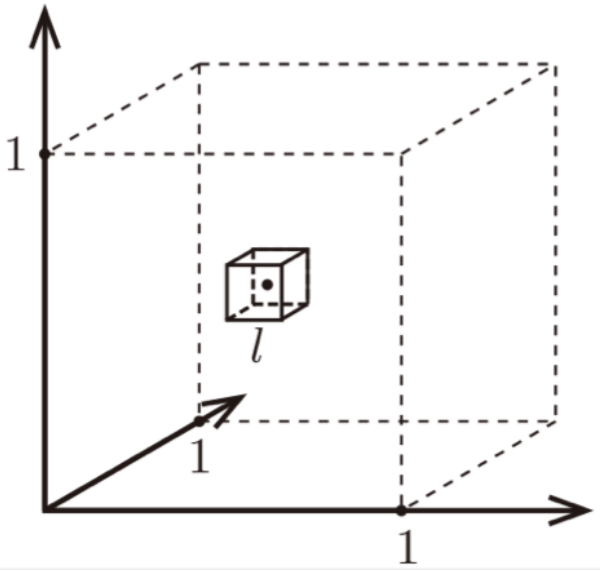
So empirically, the probability of sampling a point inside the small cube is roughly $\frac{K}{N}$

Thus, we have $l^d \approx \frac{K}{N}$

Curse of Dimensionality Intuitive Explanation (7)

Example: let us consider uniform distribution over a cube $[0, 1]^d$

We have $l^d \approx \frac{K}{N}$



$d = 3$

Q: how large we should set l, s.t., we will have K examples (out of N) fall inside the small cube?

$$l \approx \left(\frac{K}{N}\right)^{\frac{1}{d}}$$

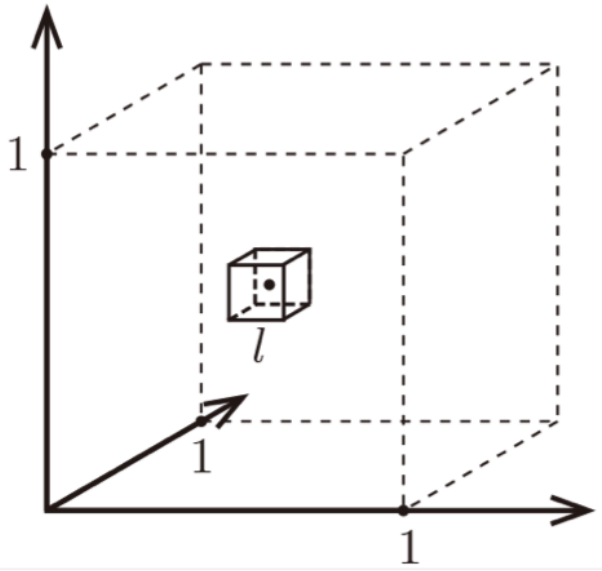
K=3

N=100

Curse of Dimensionality Intuitive Explanation (8)

Example: let us consider uniform distribution over a cube $[0, 1]^d$

We have $l^d \approx \frac{K}{N}$



$d = 3$

Q: how large we should set l, s.t., we will have K examples (out of N) fall inside the small cube?

$$l \approx \left(\frac{K}{N}\right)^{\frac{1}{d}}$$

K=3
N=100
D=1

$$l = np.power\left(\frac{3}{100}, \frac{1}{1}\right) \approx 0.03$$

K=3
N=100
D=3

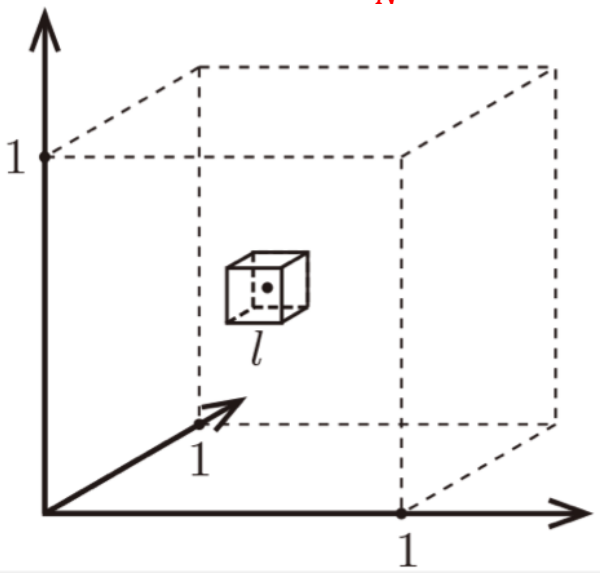
$$l = np.power\left(\frac{3}{100}, \frac{1}{3}\right) \approx 0.311$$

Curse of Dimensionality Intuitive Explanation (9)

- Key problem: in high dimensional space, points that are draw from a distribution tends to be far away from each other!

Example: let us consider uniform distribution over a cube $[0,1]^d$

We have $l^d \approx \frac{K}{N}$



$d = 3$

Q: how large we should set l, s.t., we will have K examples (out of N) fall inside the small cube?

$$l \approx \left(\frac{K}{N}\right)^{\frac{1}{d}}$$

K=3
N=100
D=10

$$l = np.power\left(\frac{3}{100}, \frac{1}{10}\right) \approx 0.704$$

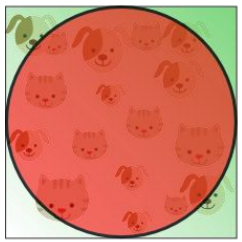
K=3
N=100
D=1000

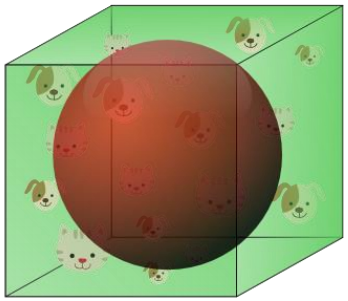
$$l = np.power\left(\frac{3}{100}, \frac{1}{1000}\right) \approx 0.9965$$

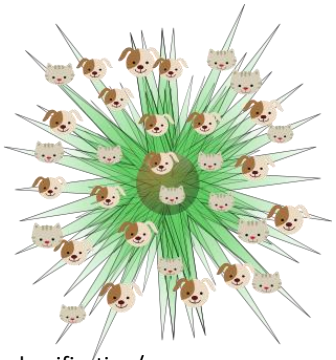
Summary of Curse of Dimensionality

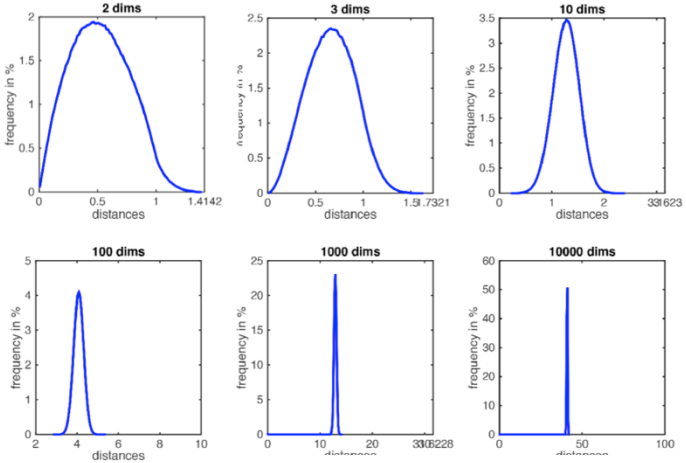
- When dimensionality increases, data becomes increasingly sparse in the space that it occupies

- Definitions of density and distance between points, which are critical for clustering and outlier detection, become less meaningful









https://www.visiondummy.com/2014/04/curse-dimensionality-affect-classification/

Distance increases as $d \rightarrow \infty$

What is Dimensionality Reduction?

Dimensionality reduction:

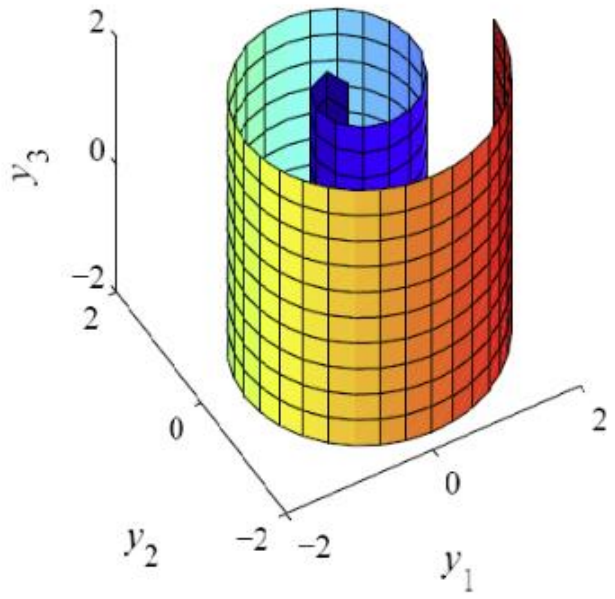
$$X \in \mathbb{R}^{n \times d} \rightarrow Z \in \mathbb{R}^{n \times k}$$

Where:

- $d \gg k$
- Preserve important structure
- Reduce noise
- Improve efficiency

Dimensionality Reduction.

- Dimensionality reduction algorithms
  - Map high-dimensional data to a lower dimension
  - While preserving structure
- They are used for
  - Visualization
  - Performance
  - Curse of dimensionality
- A ton of algorithms exist
- t-SNE is specialized for visualization
- ... and has gained a lot of popularity



Two Main Categories

I. Feature Selection

- Select subset of original features
- No transformation
- Example: Mutual Information

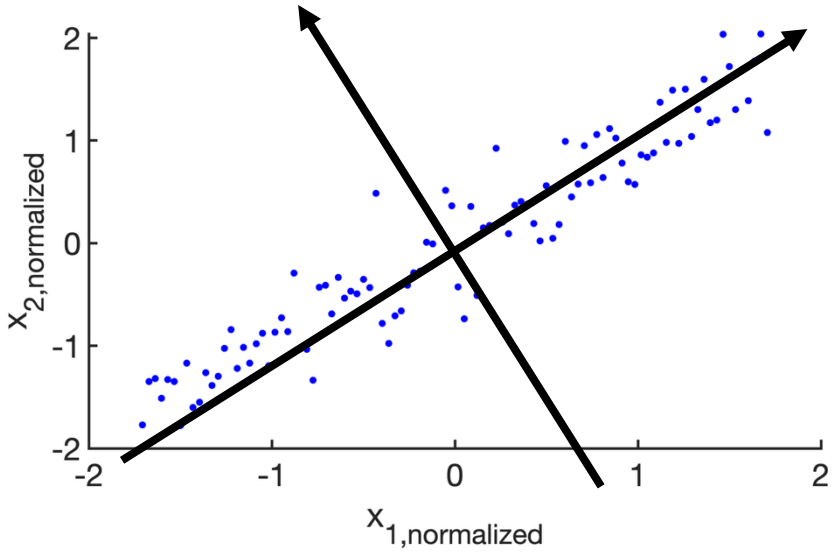
II. Feature Extraction

- Transform features into new space
- Example:
  - PCA
  - SVD
  - LDA
  - t-SNE

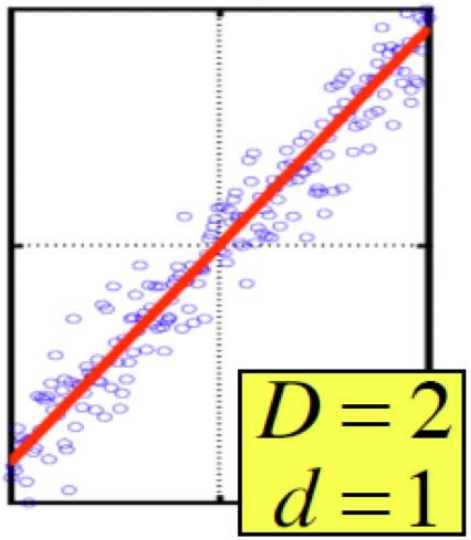
Dimensionality Reduction: PCA

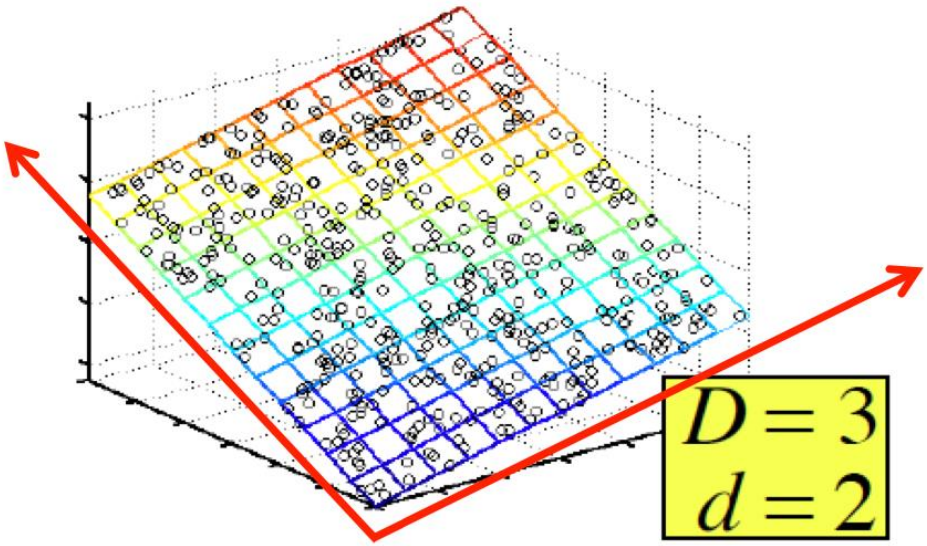
- Principal Component Analysis (PCA), is a technique used for reducing the dimensionality of data while retaining most of its variability.
- It transforms a set of possibly correlated variables into a smaller set of uncorrelated variables called principal components.

- Applications of PCA:
  - Data Visualization
  - Noise Reduction
  - Feature Reduction



PCA: Examples

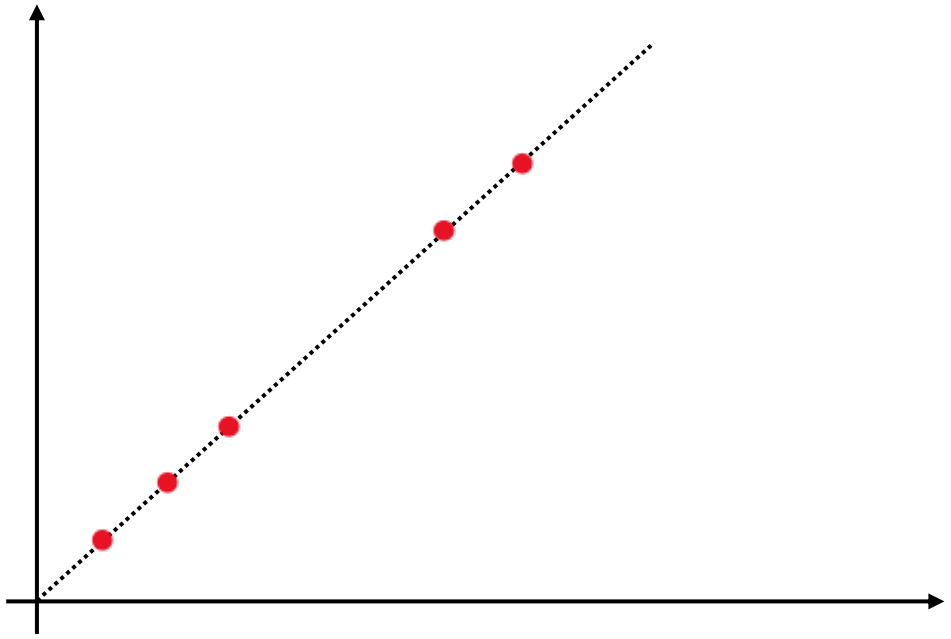




- In case where data lies on or near a low d-dimensional linear subspace, axes of this subspace are an effective representation of the data.
- Identifying the axes is known as Principal Components Analysis, and can be obtained by using classic matrix computation tools (Eigen or Singular Value Decomposition).

PCA: Intuitive Explanation (1)

	x_1	x_2
p_1	1	1
p_2	2	2
p_3	3	3
p_4	7	7
p_5	10	10



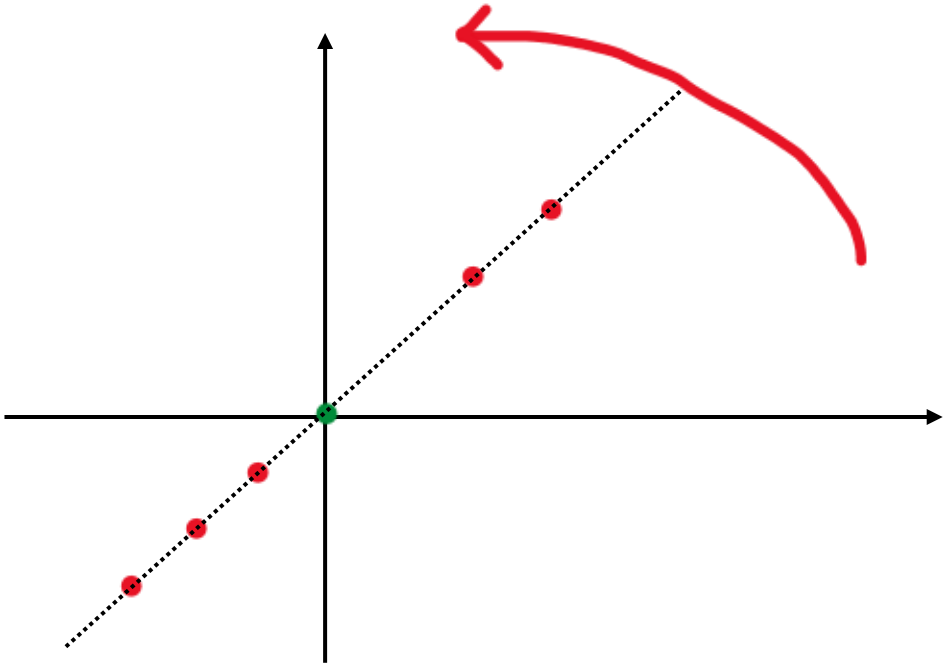
$$\overline{x_1} = \frac{1+2+3+7+10}{5} = 4.6$$

$$\overline{x_2} = \frac{1+2+3+7+10}{5} = 4.6$$

Mean-centered

PCA: Intuitive Explanation (2)

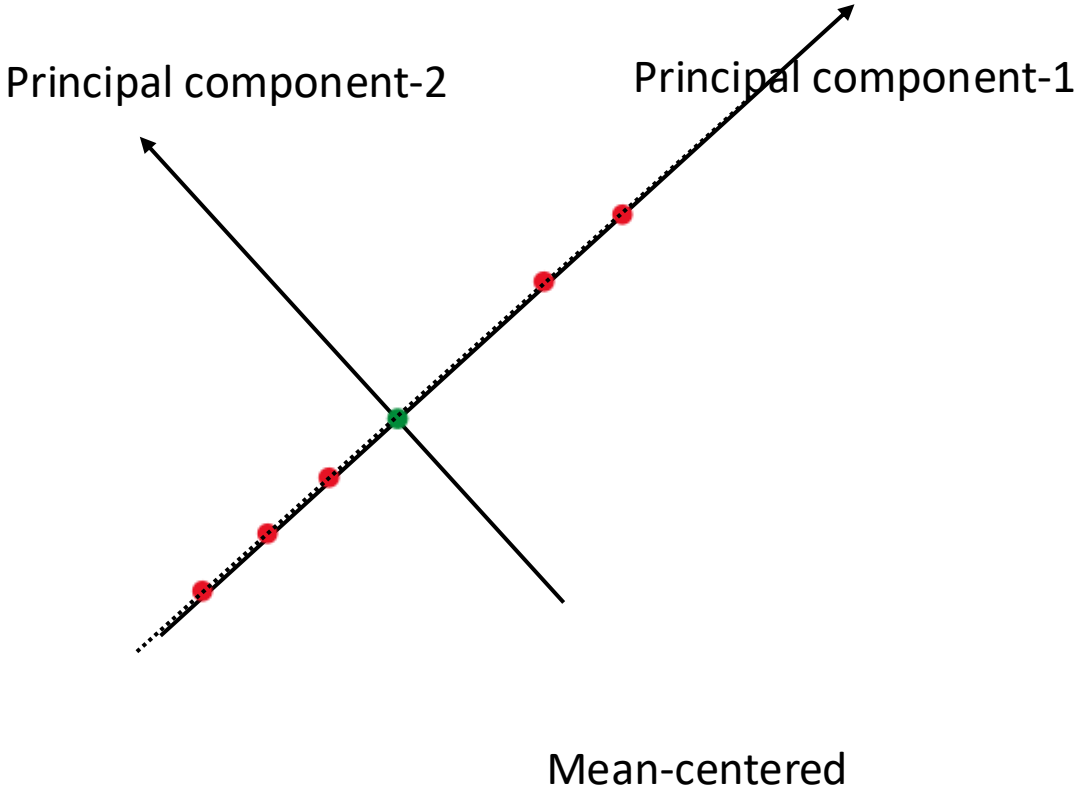
	x_1	x_2
p_1	-3.6	-3.6
p_2	-2.6	-2.6
p_3	-1.6	-1.6
p_4	2.4	2.4
p_5	5.4	5.4



Mean-centered

PCA: Intuitive Explanation (3)

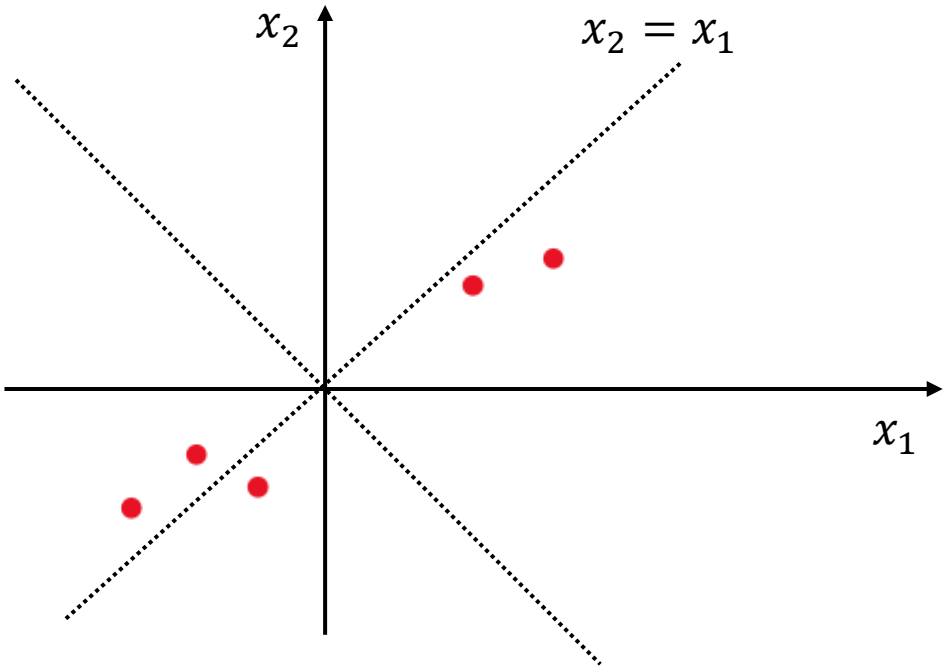
	x_1	x_2
p_1	-3.6	-3.6
p_2	-2.6	-2.6
p_3	-1.6	-1.6
p_4	2.4	2.4
p_5	5.4	5.4



PCA: Intuitive Explanation (4)

	x_1	x_2
p_1	-3.6	-2.6
p_2	-2.6	-1.6
p_3	-1.6	-2.1
p_4	2.4	1.9
p_5	5.4	4.4

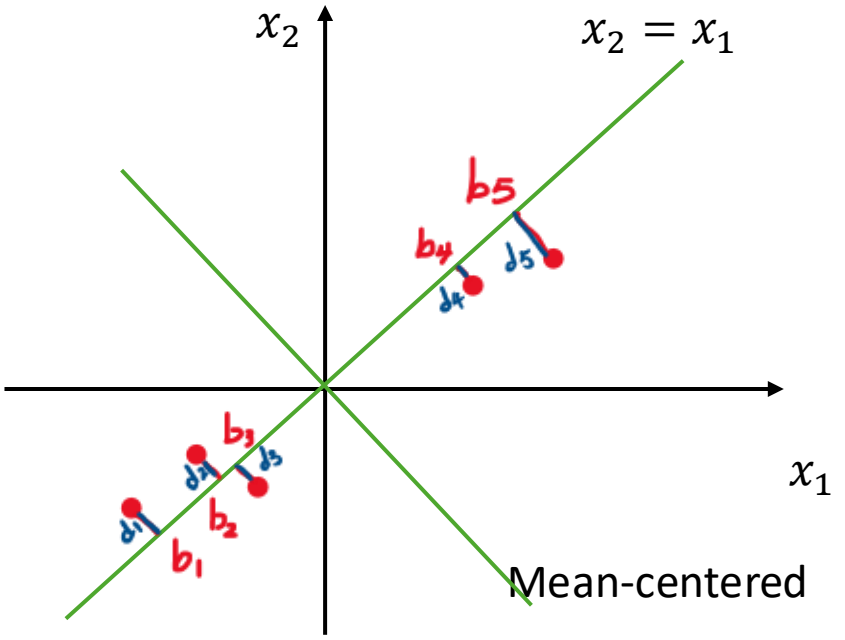
Less ideal case.



Mean-centered

PCA: Intuitive Explanation (5)

	x_1	x_2
p_1	-3.6	-2.6
p_2	-2.6	-1.6
p_3	-1.6	-2.1
p_4	2.4	1.9
p_5	5.4	4.4



Goal: find a transformation or projection with maximized variations $\sum(x_i - 0)^2$

$$b_i^2 + d_i^2 = fixed\ value$$

Maximize: $(b_1^2 + b_2^2 + b_3^2 + b_4^2 + b_5^2)$

Minimize: $(d_1^2 + d_2^2 + d_3^2 + d_4^2 + d_5^2)$

PCA illustration

- Find a projection that captures the largest amount of variation in data

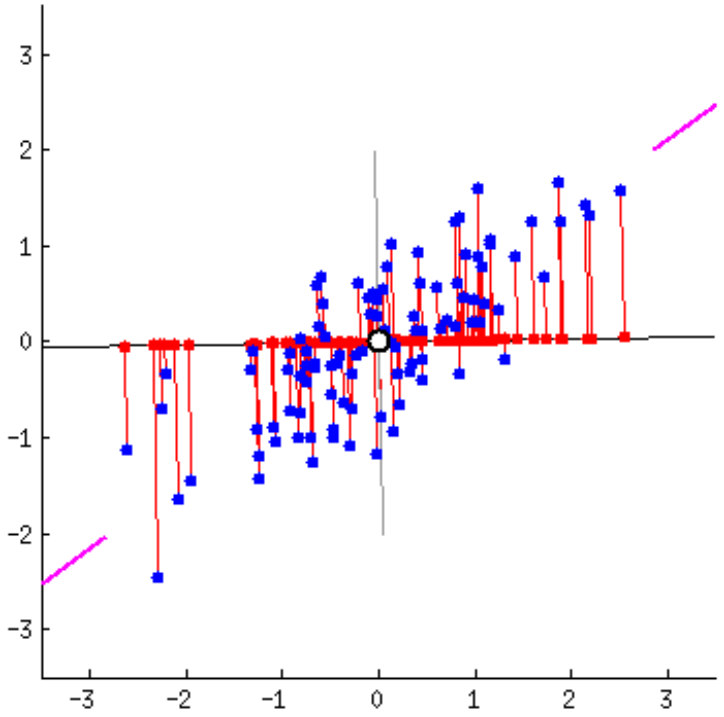


Image from https://stats.stackexchange.com/questions/134282/relationship-between-svd-and-pca-how-to-use-svd-to-perform-pca

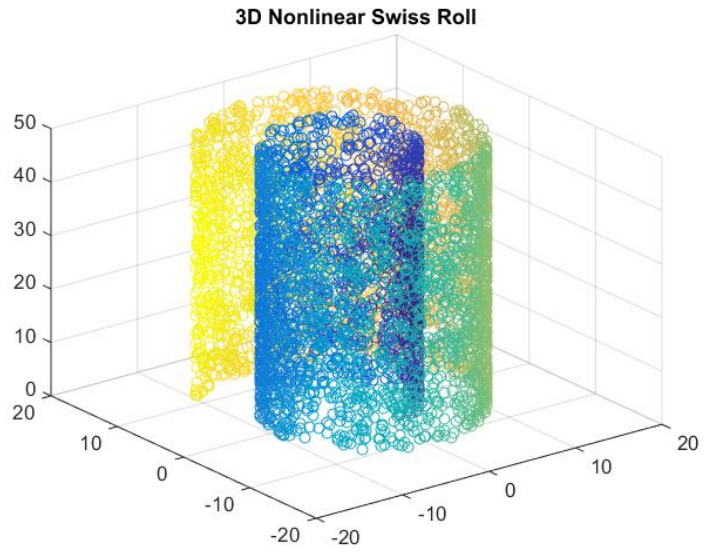
PCA Limitations

Limitation:

- Linear method
- Preserves global variance
- Cannot capture nonlinear structure

Example:

- Swiss roll dataset



Nonlinear Dimensionality Reduction

When data lies on nonlinear manifold:

Need nonlinear methods:

- Isomap
- LLE
- Kernel PCA
- t-SNE

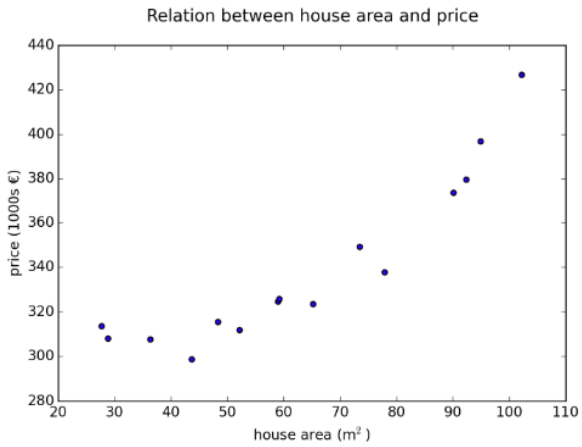
Visualization is key to understand data easily (1)

Data of house areas in m^2 and price in 1000s of euros.

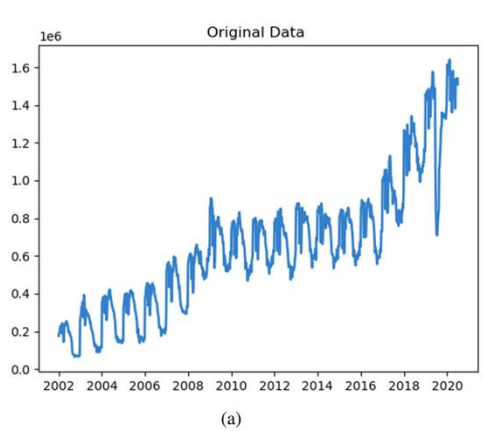
Area	price
43.69	298.71
28.82	308.
102.22	426.68
36.32	307.53
48.35	315.4

Area	price
59.04	324.48
90.13	373.8
59.24	325.71
94.89	396.69
27.72	313.53

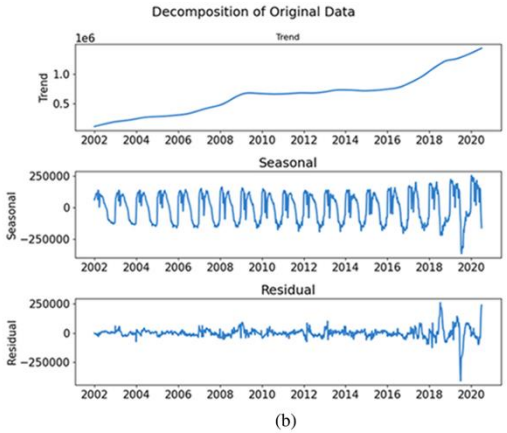
Area	price
65.2	323.43
92.38	379.56
77.86	337.77
73.48	349.15
52.19	311.86



Tabular data



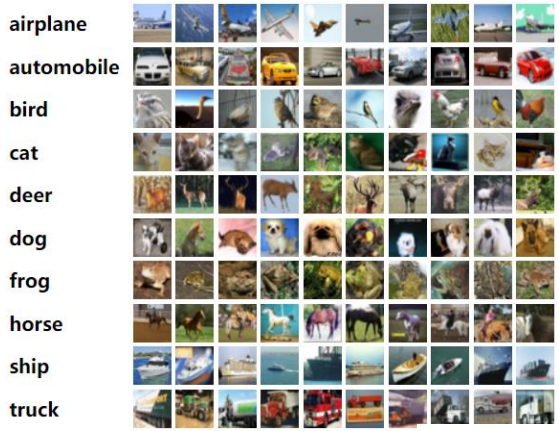
(a)



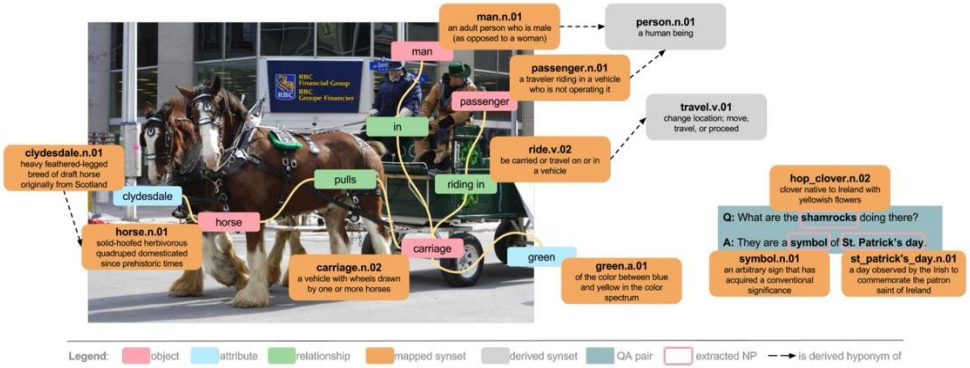
(b)

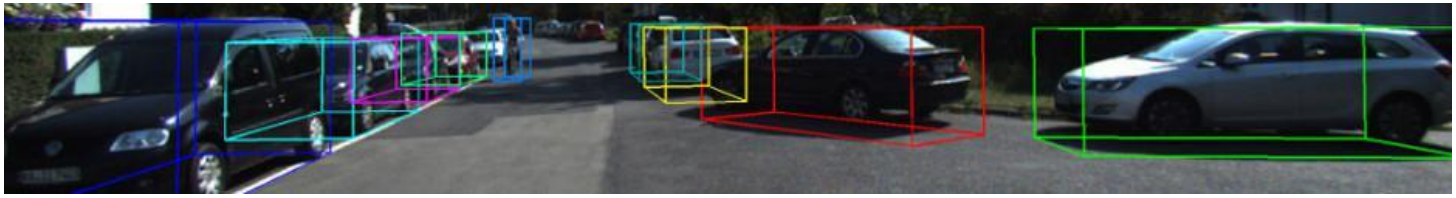
Time-Series

Visualization is key to understand data easily (2)

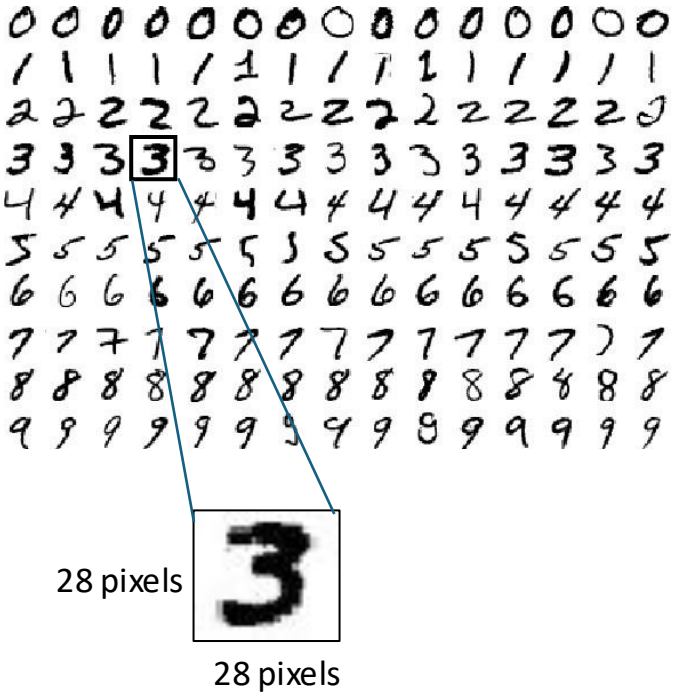




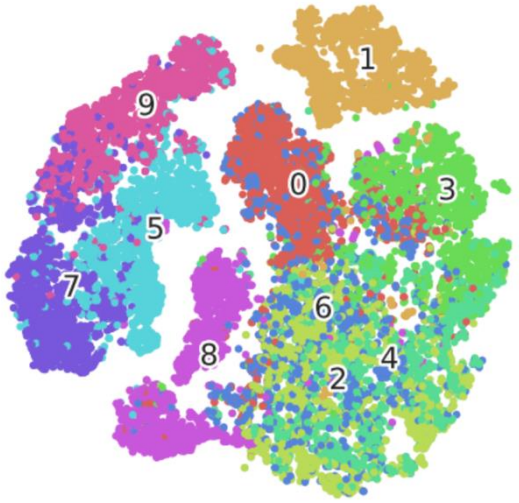




Visualization is key to understand data easily (3)



What we want:



$x^{(i)}$: 784-dimensional observation

$z^{(i)}$: 2-dimensional observation

Visualization is key to understand data easily (4)

	Attr_1	Attr_2	…	Attr_1000
x_1	0.3	1.2	…	4.3
x_2	3.2	1.9	…	2.7
x_3	2.8	1.3	…	5.2
⋮	⋮	⋮	⋮	⋮
x_n	0.5	2.1	…	3.7

How ??

Data Visualization Gap

- **(Problem) Visualization Gap:** We can only see in 2D or 3D. How do we visualize a 784-dimensional vector?

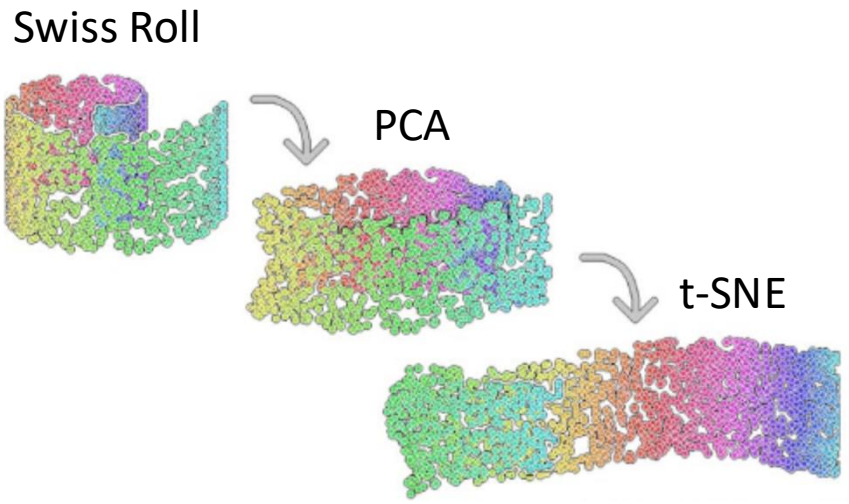
- **(We want) Structure Preservation:** We need to project data down while keeping similar points close together.

- **(Observation) Complex Manifolds:** Data often lies on a curved surface (manifold) within the high-dimensional space.

Linear vs. Non-Linear

Principal Component Analysis (PCA)

PCA is linear. It rotates the data to maximize variance. It's great for preserving global structure but often fails to separate complex, non-linear clusters.



t-SNE

t-SNE is non-linear. It focuses on preserving **local neighborhoods**. It can unwrap complex manifolds (like the Swiss Roll) to reveal distinct clusters.

Stochastic Neighbor Embedding (SNE)

SNE basic idea:

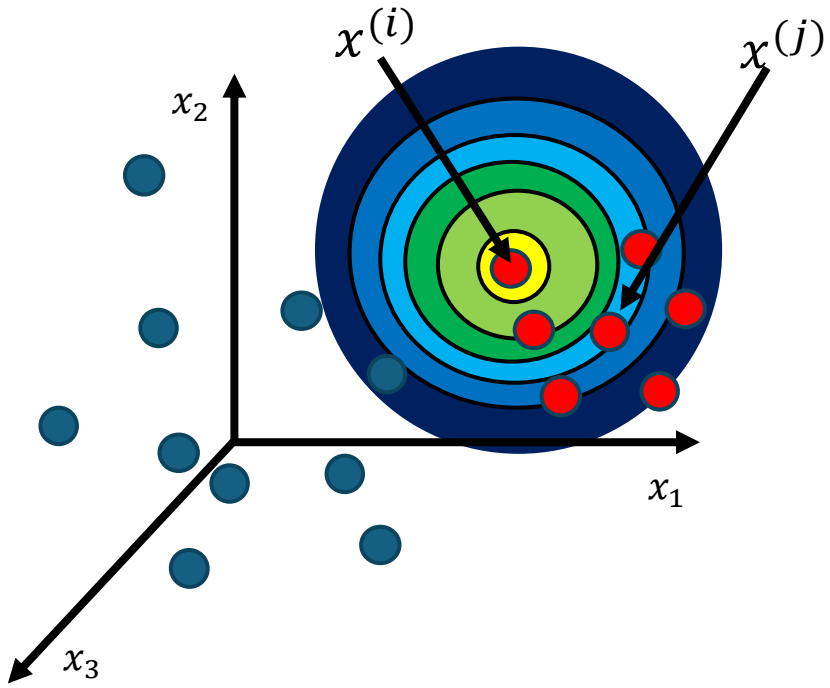
- ”Encode” high dimensional neighborhood information as a distribution
- Intuition: Random walk between data points.
  - High probability to jump to a close point
- Find low dimensional points such that their neighborhood distribution is *similar*.

How do you measure distance between distributions?

- Most common measure: KL divergence

Step-1: High-Dimensional Similarity

SNE converts Euclidean distance to similarities, that can be interpreted as probabilities.



A point closer to $x^{(i)}$ will be more likely than one further away.

1. Consider one observation $x^{(i)}$
2. Construct a Gaussian distribution P_i

centered on $x^{(i)}$ with a standard deviation σ_i

σ_i: the size of the neighborhood

- Very low : all the probability is in the nearest neighbor
- Very high : Uniform weights

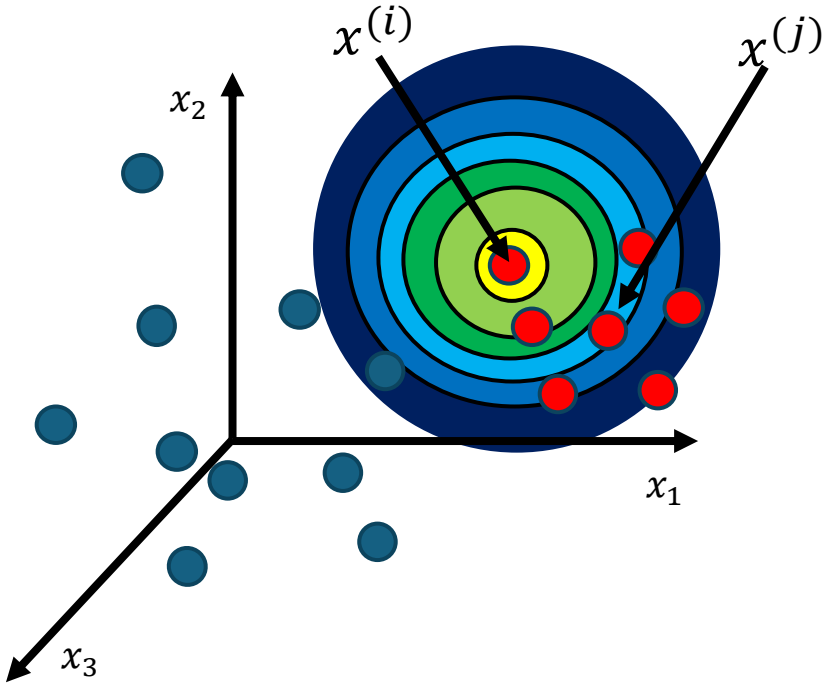
3. Calculate the conditional probability

$p_{j|i} = P_i(x^{(j)})$ for all other points $x^{(j)}$

= the probability that point $x^{(i)}$ chooses $x^{(j)}$ as its neighbor

$$p_{j|i} = \frac{e^{\left(-\frac{\left|\left|x^{(i)}-x^{(j)}\right|\right|^2}{2\sigma_i^2}\right)}}{\sum_{k\neq i} e^{\left(-\left|\left|x^{(i)}-x^{(k)}\right|\right|^2/2\sigma_i^2\right)}}$$

Step-1: High-Dimensional Similarity (Cont'd)



1. Consider one observation $x^{(i)}$
2. Construct a Gaussian distribution P_i centered on $x^{(i)}$ with a standard deviation σ_i

3. Calculate the conditional probability $p_{j|i} = P_i(x^{(j)})$ for all other points $x^{(j)}$

Point	$x^{(1)}$	$x^{(2)}$	..	$x^{(m)}$
$x^{(1)}$	0	0.18	…	0.56
$x^{(2)}$	0.2	0	…	0.24
…	…	…	…	…
$x^{(m)}$	0.49	0.16	…	0

P_i :A similarity matrix based on probability

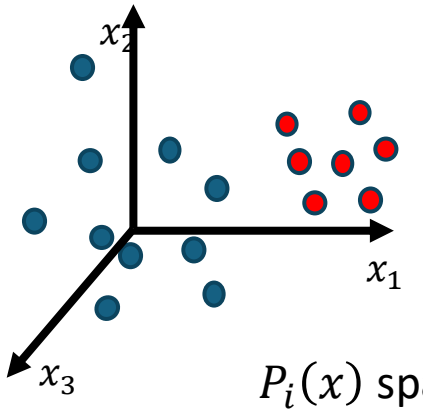
Step 2: Low-Dimensional Embedding

$$p_{j|i} = \frac{e^{\left(-\frac{\left||x^{(i)}-x^{(j)}\right||^2}{2\sigma_i^2}\right)}}{\sum_{k\neq i} e^{\left(-\left||x^{(i)}-x^{(k)}\right||^2/2\sigma_i^2\right)}}$$

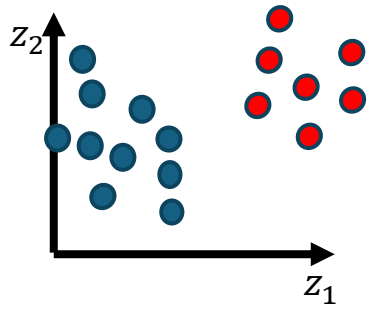
$$q_{i|j} = \frac{\left(1+\left||z^{(i)}-z^{(j)}\right||^2\right)^{-1}}{\sum_{k\neq l}\left(1+\left||z^{(k)}-z^{(l)}\right||^2\right)^{-1}}$$

Student's t-distribution

Data space



$P_i(x)$ space

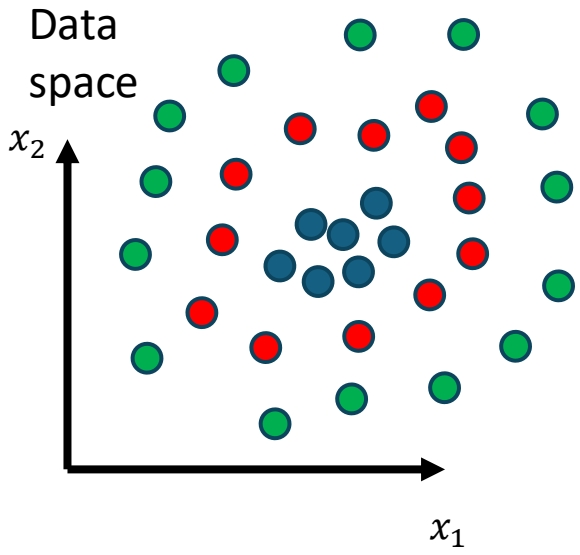


$Q_i(z)$ space

Map space

Goal of SNE: Arrange points in z such that $Q \approx P$

Why 't'? The Crowding Problem





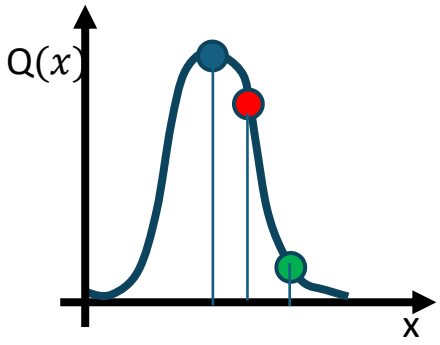
What you'd like:



What SNE does:



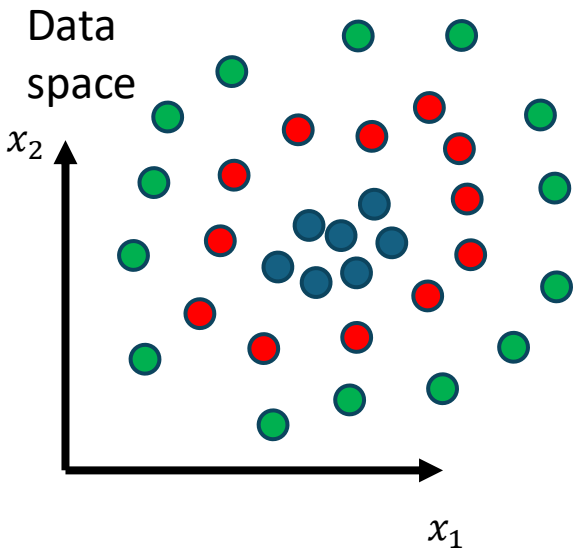
Why? Think how the Gaussian distribution looks:



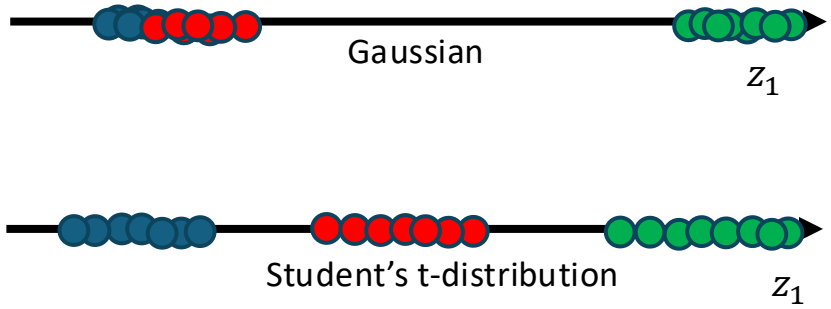
Exponential drop-off
Hence moderately close becomes **too close** and moderately far becomes **too far**

Why 't'? The Crowding Problem (Cont'd)

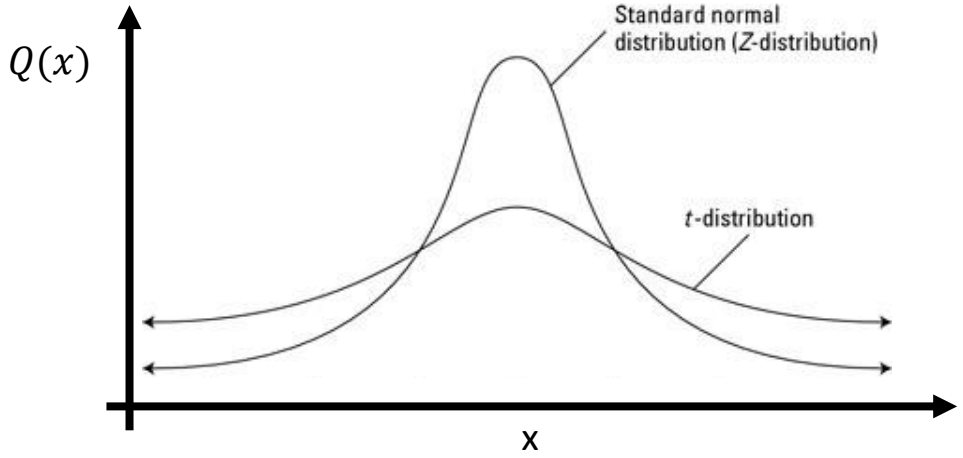
Issue: Crowding of points



2D—>1D



Solution: Use a Q with a fatter tail than Gaussian such as the **Student's t-distribution**



t-SNE

t

Student's t-distribution

S

Stochastic
- Probabilistic method
- Different results each time

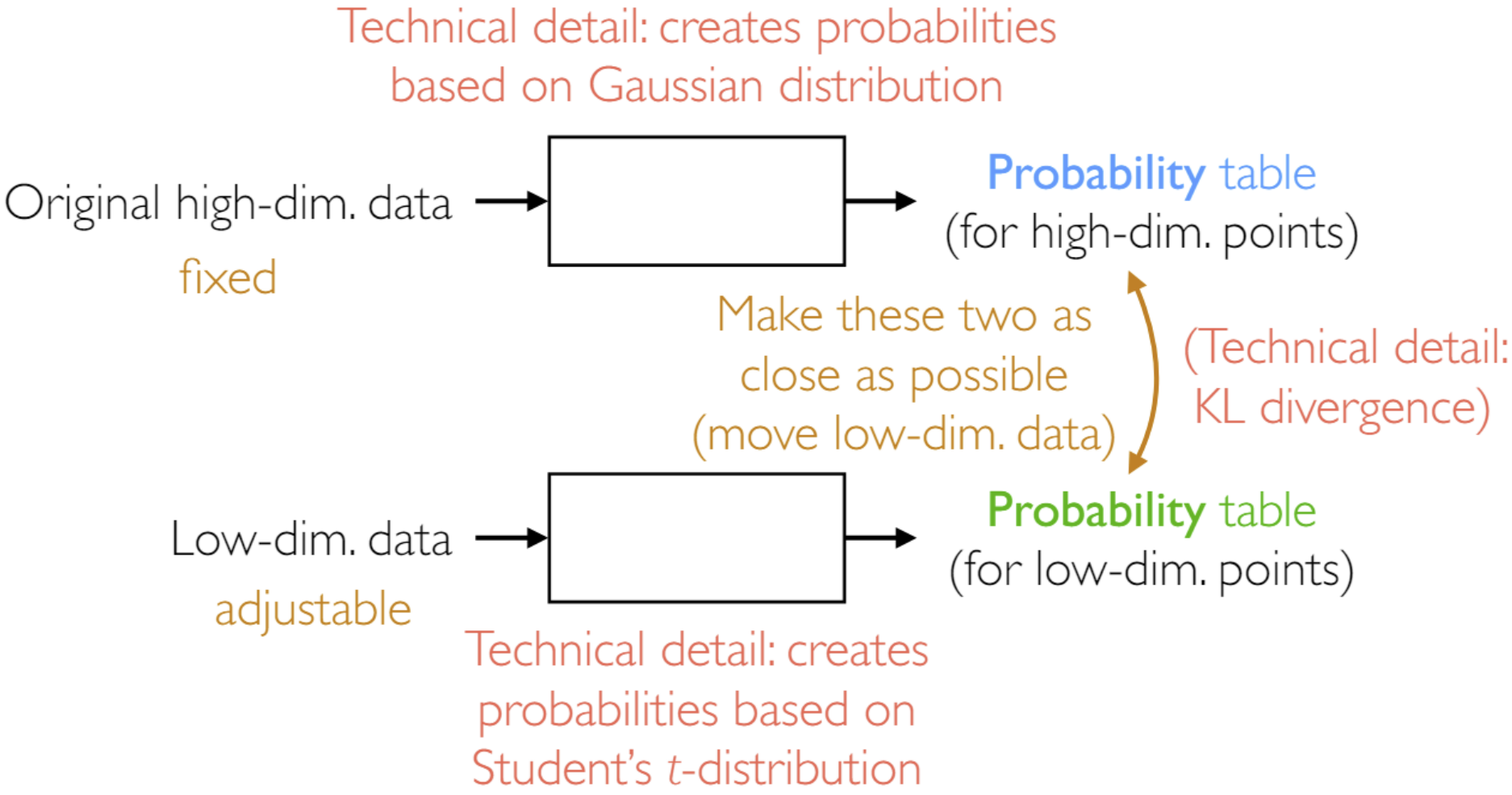
N

Neighborhood
- Preserves local structure

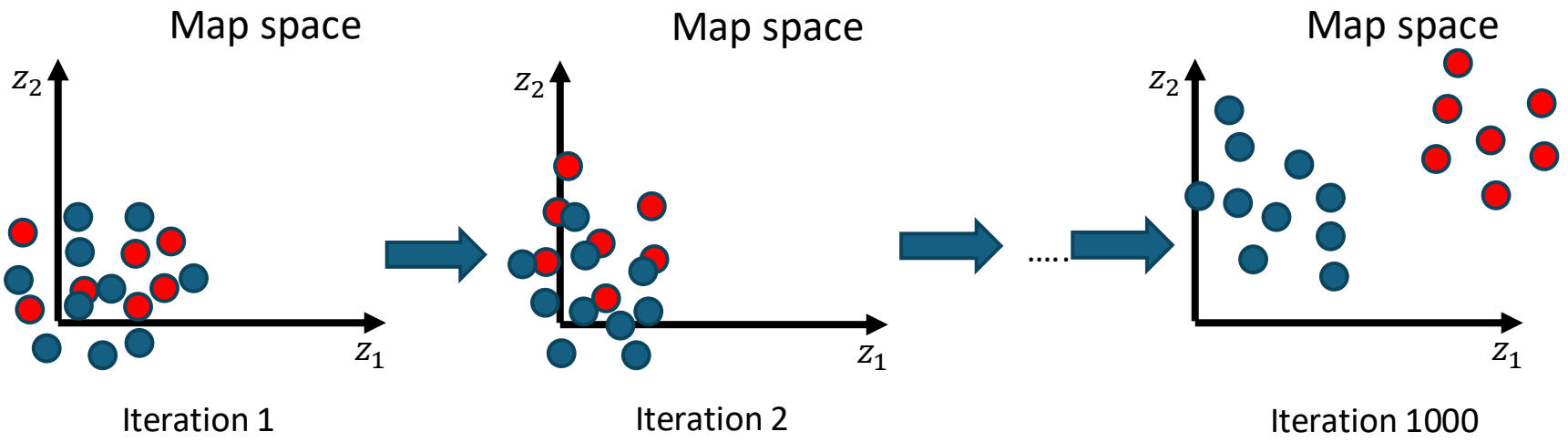
E

Embedding
- Dimensionality reduction
- In this case, to 2D (sometimes 3D)

SNE problem formulation : project x to z



SNE explanation



$Q_i(z)$ space

We want both spaces equivalent probability distributions $P_i \approx Q_i$ for all the points i

SNE: An iterative algorithm that tries to arrange points so that $P_i \approx Q_i$

Step 3: Optimization

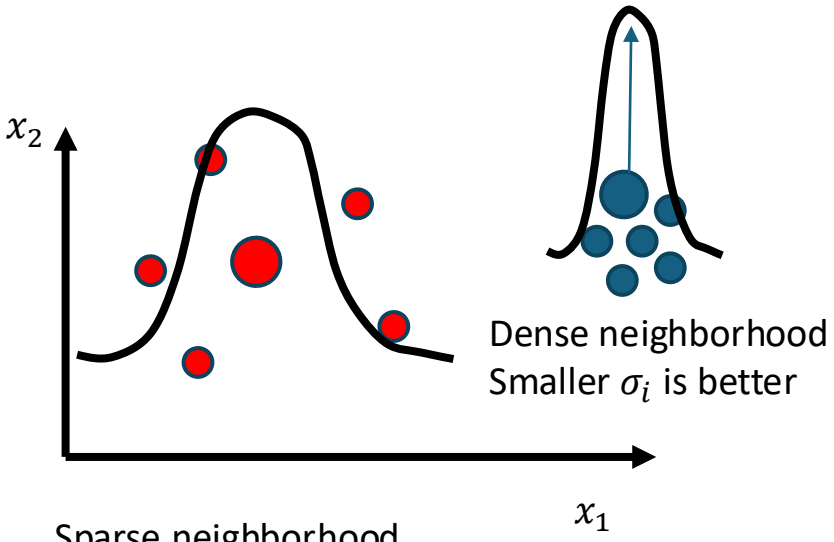
- We want the low-dimensional probabilities (Q) to match the high-dimensional probabilities (P) as closely as possible.
- We minimize the **Kullback-Leibler (KL) Divergence** cost function using Gradient Descent.

$$\text{Cost} = KL(P||Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

Gradient Descent

- Start with random points in 2D.
- Calculate the gradient (forces pulling similar points together and repelling dissimilar ones).
- Update positions iteratively.

Hyperparameter: Perplexity



Parameter σ_i should account for the size of neighborhoods:

- Very low σ_i: all the probability is in the nearest neighbor
- Very high σ_i: uniform weights

Perplexity, a smooth measure of the # of neighbors: (perplexity π which is a measure of ***average number of neighbors of each point)***

$$\pi = 2^{-\sum_j p_{j|i} log_2 p_{j|i}}$$

-- Combining this with the expression for $p_{j|i}$, we can compute the proper σ_i for each point

-- π between 5 to 50 typically work well for most problems

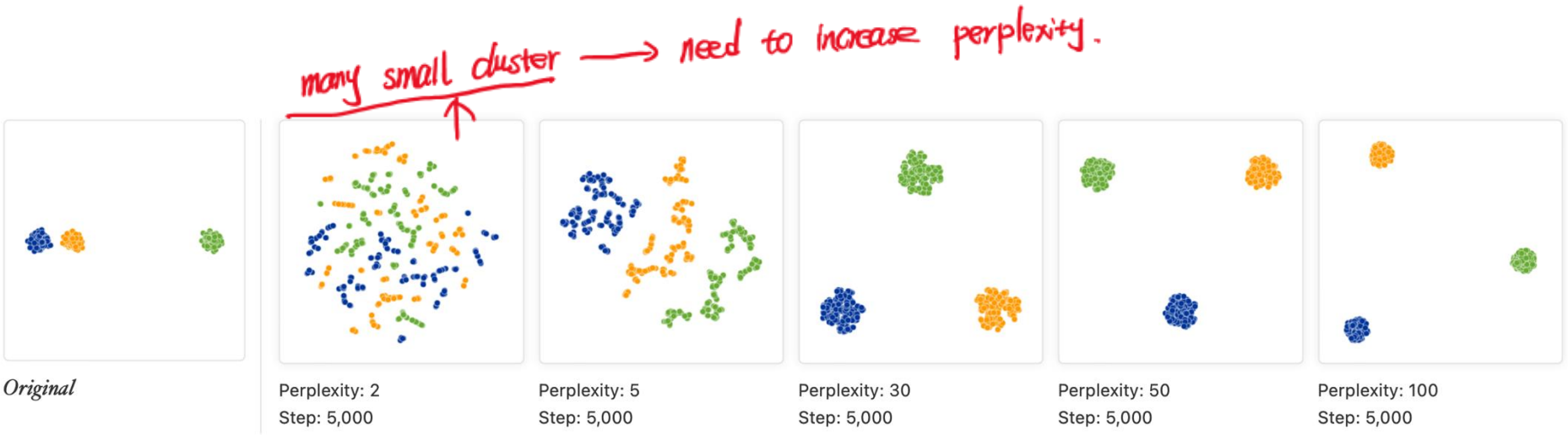
$\pi = 2^{H(P_i)}$, $H(P_i) = -\sum_j P_{j|i} \log_2 P_{j|i}$

π usually is given. Let's say $\pi = 50$.

→ dense area → small σ
→ spare area → large σ

dynamically balance the dense and spare areas by adjusting σ value.

Hyperparameter: Perplexity (Cont'd)



https://distill.pub/2016/misread-tsne/

- **Small perplexity**: focus more on local structure, tends to preserve data relationship within a small range. which may lead to over-segmentation of dusters.
- **large perplexity**: focus more on global structure, tends to merge data points in sparse region, which may lead to over-merging of clusters.

Case Study: MNIST

The "Hello World" of Machine Learning. Here, t-SNE has taken 784-pixel images of handwritten digits and mapped them to 2D.

Notice how it naturally grouped the digits 0-9 into distinct, separated clusters without ever being told the labels.



t-SNE vs PCA

Feature	PCA	t-SNE
Type	Linear	Non-linear
Goal	Preserve **global structure** and maximize variance	Preserve **local structure** (neighborhood relationships)
Method	Projects data onto a lower-dimensional space using **eigenvectors** (linear algebra)	Uses **probability distributions** and minimizes divergence (KL-divergence) between high- and low-dimensional similarities
Speed	Fast, scalable to large datasets	Slow, not scalable (especially for large datasets)
Reproducibility	Deterministic (always gives same result)	Non-deterministic (results may vary unless random seed is fixed)
Output Dimensionality	Can reduce to any number of dimensions	Typically used for 2D or 3D visualization only
Use Case	Preprocessing, noise reduction, feature engineering	Data **visualization**, especially for high-dimensional data like images or text embeddings

t-SNE summary

- T-SNE is a great way to visualize data.
- Preserves ***both*** local (always) and global (usually but not always) structure in the data, thus provides better visualizations than PCA, which only preserves global structure in the data.
- Help understand "black-box" algorithms like CNN.
- Reduced "crowding problem" with heavy tailed distribution.
- Non-convex optimization – solved by GD (use multiple restarts, choose the best results)
- Can use PCA first to project the original feature into lower feature space, then use t-SNE.

Review

- Data and Data Preprocessing
- Model Evaluation
- Classifiers
  - Bayes Classifier
  - Decision Tree
  - Perceptron
  - Logistic Regression
  - SVM
- Ensemble Method
  - Bagging
  - Boosting
- Clustering
  - K-Means
  - GMM
- Dimensionality Reduction
  - t-SNE

Data & Data Preprocessing

- Data
  - types of attributes
  - Common Data Types
  - Data Quality Issues
- Data Preprocessing
  - Common data preprocessing steps
  - Pipeline

Model Evaluation

- Common metrics for model evaluation
  - Precision, recall, accuracy, F1-score, confusion matrix
  - AUC, ROC
  - Given a scenario, which metric should we use?
- Model evaluation strategies
  - Cross-validation
  - LOOCV
  - Overfitting vs. Underfitting

Classifiers (1)

- Bayes Classifier
  - Use it to solve a toy example.
- Decision Tree
  - Uses metrics like Gini Impurity or Entropy (Information Gain) to choose the best feature to split on.
  - How to avoid overfitting
- Perceptron
  - Loss function
  - Gradient Descent Algorithm
  - Use GD to solve a toy example.
  - Limitations

Classifiers (2)

- Logistic Regression
  - From perceptron to logistic regression (Motivation?)
  - Compare Logistic regression to Perceptron.
  - Hand-calculating a Logistic regression (with GD) on a toy dataset.
- SVM
  - Motivation
  - Hard-SVM and Soft-SVM (concepts, loss function)
  - Kernel-tricks
  - Hand-calculating a Hard-SVM on a toy dataset.

Ensemble Method

Ensemble Method:

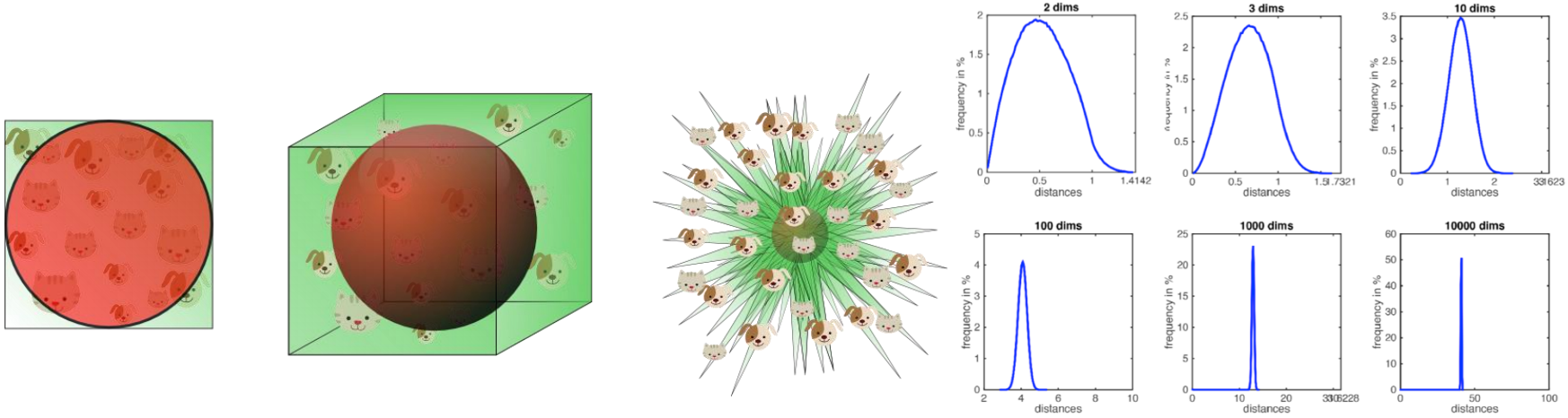
- Core ideas of Bagging and Boosting
- Famous algorithms:
  - Random Forest,
  - Adaboost
- Bias-Variance Perspective
- Bagging vs. Boosting (difference, and given a scenario, which method is more appropriate?)

Clustering

- K-means
  - Algorithms
  - Limitations (random initialization, sensitive to outliers, K values)
  - How to address the limitations
- GMM
  - Comparison between K-means and GMM
  - Learned parameters
  - Generative model
  - How to use it in real applications

Dimensionality Reduction

- Curse of Dimensionality



- PCA and t-SNE